IS6003 · ĐỒ ÁN CUỐI KỲ

Rút ngắn thời gian phát triển ứng dụng

Cách làm, từng bước: mô phỏng quy trình, dùng ML tìm nguyên nhân, thiết kế lại và kiểm chứng bằng mô phỏng

132,5
ngày làm việc, cycle time trung bình hiện tại

≥ 20%
mục tiêu rút ngắn

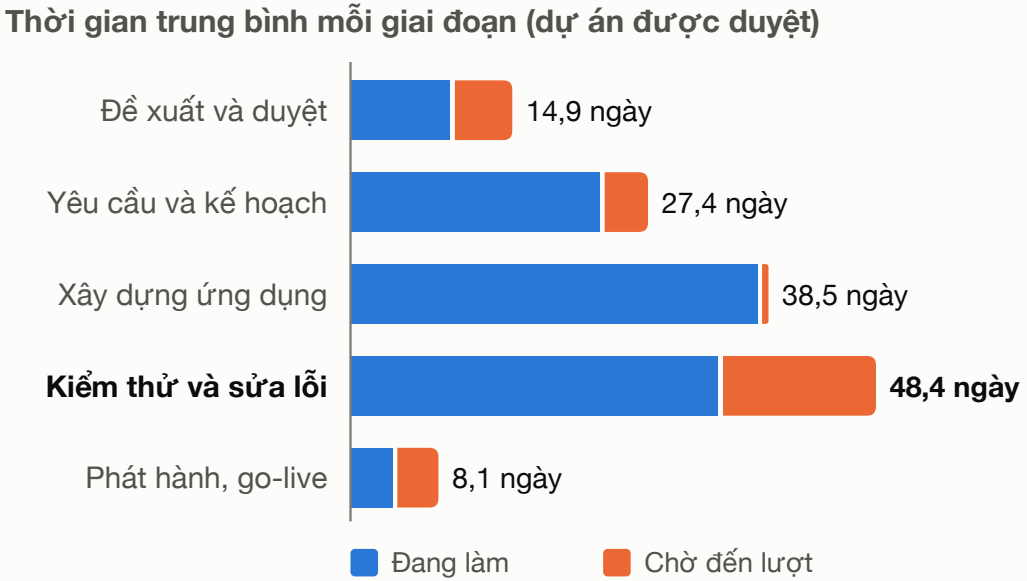
27,8%
rút ngắn với thiết kế tốt nhất (C1 + C2 + C4)

Phương pháp: DSRM (Peffers et al., 2007). Quy trình theo *Fundamentals of BPM*, 2nd ed. (Dumas et al.). Mọi số liệu là mô phỏng, không phải dữ liệu thật.

Vấn đề: một yêu cầu mất khoảng 6 tháng, và nhiều nhất ở vòng kiểm thử

Một yêu cầu đi qua: 3 lần duyệt, phân tích yêu cầu, xây dựng, **4 bước kiểm thử nối tiếp ở QC**, rồi triển khai và go-live. 5 team chuyển việc cho nhau.

- Cycle time trung bình: **132,5 ngày** (trung vị 122,9, 10% chậm nhất trên 198 ngày)
- **37,3%** dự án bị trả về ở bước kiểm thử ít nhất 1 lần
- Mỗi lần trả về, cả 4 bước kiểm thử chạy lại từ đầu
- QC là team bận nhất: **87%** thời gian



Ý chính: Kiểm thử và sửa lỗi chiếm 48 trên 137 ngày, và đây là chỗ chờ lâu nhất. Đó là nơi cần sửa.

Mục tiêu: rút ngắn ít nhất 20%, và phải chứng minh được

> Giảm cycle time trung bình **ít nhất 20%** so với as-is mô phỏng.
> Khoảng tin cậy 95% của phần giảm phải **lớn hơn 0**.

- **Cycle time** = từ lúc nhận yêu cầu đến khi xong bước "Đưa vào vận hành" (go-live)
- Đơn vị: ngày làm việc
- Theo dõi vận hành và đánh giá dự án vẫn nằm trong mô hình, nhưng không tính vào cycle time
- As-is và to-be chạy trên cùng tham số, cùng lượng yêu cầu, cùng chuỗi số ngẫu nhiên

Câu hỏi nghiên cứu

Thay đổi quy trình nào làm cycle time ngắn nhất, và mỗi thay đổi đóng góp bao nhiêu?

Vì sao cần CI > 0

Mô phỏng có yếu tố ngẫu nhiên. Nếu khoảng tin cậy chứa 0, có thể phần giảm chỉ là may mắn.

Ý chính: Mục tiêu đo được, có điều kiện thống kê, và mọi kịch bản được so trên cùng một nền.

Cách làm: 8 bước, theo DSRM

0 — Phase 0

Sửa mô hình

Áp luật vẽ BPMN của sách (7PMG) để máy chạy được

1 — DSRM 1–2

Phân tích định tính

Value-added, lãng phí, danh sách vấn đề, đặt mục tiêu

2 — Phase 2

Sinh dữ liệu

Simulator chạy mô hình, ghi event log 2.000 dự án

3 — Phase 3

Đo baseline

Cycle time, thời gian chờ, tỷ lệ làm lại

4 — Phase 4

ML chẩn đoán

Dự đoán cycle time và rework, SHAP xếp hạng nguyên nhân

5 — DSRM 3

Thiết kế lại

4 thay đổi C1–C4, mỗi cái theo 1 heuristic của sách

6 — DSRM 4–5

Đánh giá

16 kịch bản × 30 lần chạy, ablation

7 — DSRM 6

Trình bày

Notebook, trang web, demo

Dữ liệu → **chậm ở đâu** (baseline) → **vì sao chậm** (ML) → **sửa gì** (redesign) → **sửa có hiệu quả không** (mô phỏng)

Ý chính: DSRM: phương pháp nghiên cứu thiết kế gồm 6 bước: vấn đề, mục tiêu, thiết kế, minh hoạ, đánh giá, trình bày.

Phần nào thật, phần nào giả

Loại	Gồm những gì	Mức tin cậy
Gần thật	Cấu trúc quy trình: các bước, thứ tự, ai làm, chỗ nào bị trả về (`as_is.bpmn`)	Cao Bạn mô tả nó
Đúng về phương pháp	Công thức của sách, cách mô phỏng, cách chia dữ liệu ML, cách tính khoảng tin cậy	Cao Đã kiểm tra chéo
Giả định	Mọi con số đầu vào: số yêu cầu mỗi năm, mỗi bước mất mấy ngày, xác suất bị trả về, QC có mấy người, độ phức tạp làm chậm bao nhiêu	Thấp Tự ước lượng, ghi hết trong `as_is.params.toml`
Giả định của redesign	Kiểm thử lại mất ~2 ngày, C3 giảm tỷ lệ lỗi từ 35% xuống 27%	Thấp
Kết quả	132,5 ngày, 27,8%, thứ hạng nguyên nhân…	Tính ra từ giả định

> Kết quả không phải số **đo được**. Nó là câu **"nếu… thì…"**: nếu quy trình chạy với các giả định này, thì thiết kế mới nhanh hơn 27,8%.

Ý chính: Không có dữ liệu thật của công ty. Vì vậy mỗi tham số đều có lý do ghi kèm, và hiển thị trong bảng giả định.

Mô phỏng hoạt động thế nào

1. Yêu cầu đến

Trung bình 0,234 yêu cầu mỗi ngày làm việc (~58 mỗi năm), thời điểm ngẫu nhiên.

Mỗi yêu cầu rút ngẫu nhiên 6 thuộc tính: độ phức tạp (1–3), mức bảo mật (1–3), số hệ thống tích hợp, loại dự án, độ bận của team, đơn vị yêu cầu.

2. Đi qua mô hình

Mỗi yêu cầu là một **token** đi theo đúng file `as_is.bpmn`.

Mỗi lane là một nhóm người có số lượng cố định (QC: 8, Phát triển: 13). Hết người rảnh thì phải **chờ**.

Thời gian mỗi bước: phân phối tam giác (ngắn nhất, hay gặp nhất, dài nhất), ví dụ xây dựng 8 / 14 / 25 ngày, +40% mỗi bậc phức tạp.

3. Tại cổng XOR

Xác suất bị trả về phụ thuộc thuộc tính (hàm logistic). Ở bước kiểm thử:

App đơn giản	**~17%**
App phức tạp, bảo mật cao	**~65%**

Bỏ 250 ngày đầu (warm-up), vì lúc đầu hệ thống còn trống.

Kết quả: một **event log** theo mẫu của sách. Mỗi dòng là một bước của một dự án, có thời điểm sẵn sàng, bắt đầu, kết thúc, lane và thuộc tính. Chờ = sẵn sàng → bắt đầu. Xử lý = bắt đầu → kết thúc.

Ý chính: Discrete-event simulation: thời gian nhảy từ sự kiện này sang sự kiện kế. Mô hình bạn vẽ chính là thứ được chạy.

Làm sao biết simulator tính đúng

Kiểm tra 1: so với công thức của sách

Sách tính cycle time lý thuyết bằng flow analysis (§7.1). Ta chạy simulator với số người không giới hạn để không có hàng đợi, rồi so hai kết quả.

Công thức của sách	**105,4 ngày**
Simulator, không hàng đợi	**104,7 ngày**

Hai cách tính độc lập ra gần bằng nhau, nên simulator tính đúng.

Kiểm tra 2: hiệu ứng đã cài trước

Ta ghi lại chính xác hiệu ứng nào đã cài vào dữ liệu (`as_is_planted_effects.md`), ví dụ "độ phức tạp làm tăng lỗi".

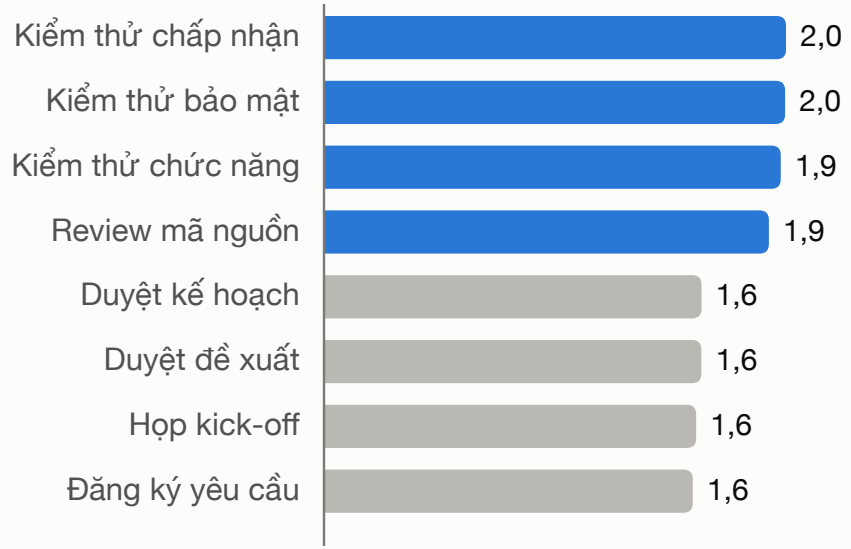
Đơn vị yêu cầu cố tình **không** ảnh hưởng gì. Đây là *biến đối chứng*: nếu ML xếp nó cao thì ML đang học nhiễu.

Sau đó kiểm tra ML có tìm lại đúng hay không (xem phần ML).

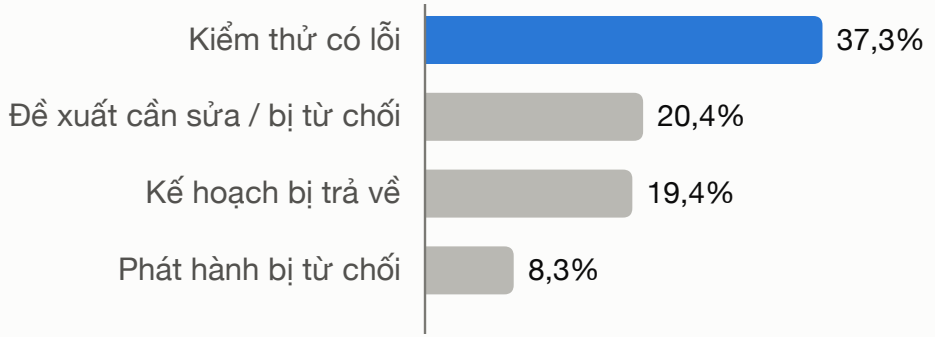
> Hai phép kiểm tra này chứng minh **code đúng**. Chúng không chứng minh **giả định đúng**.

Baseline: điểm nghẽn là vòng kiểm thử ở QC

Thời gian chờ trung bình mỗi lần chạy bước (ngày), 8 bước chờ lâu nhất



Tỷ lệ dự án bị trả về ít nhất 1 lần (%)



- 4 bước chờ lâu nhất đều ở QC (màu xanh)
- Một vòng kiểm thử ≈ 18 ngày làm + ~8 ngày chờ ≈ **26 ngày**
- Chờ chiếm 21% cycle time. Thực tế thường cao hơn nhiều, vì mô phỏng không có trễ bàn giao, nghỉ lễ hay người làm nhiều dự án cùng lúc

Ý chính: Vòng kiểm thử vừa dài, vừa hay lặp lại, và mỗi lần lặp lại phải xếp hàng ở QC.

ML: làm gì và làm thế nào

Hai bài toán

A. Regression: dự đoán cycle time ngay khi yêu cầu đến.

B. Classification: ở mỗi cổng có thể trả về, dự đoán dự án có bị trả về không.

Đầu vào: chỉ 6 thuộc tính biết được lúc yêu cầu đến.

Ba mức model

1. Mốc so sánh: đoán trung bình, hoặc đoán lớp phổ biến nhất.
2. Linear / logistic regression.
3. Gradient boosting.

Chia dữ liệu theo thời gian

80% dự án đến trước để train, 20% đến sau để test. Chia ngẫu nhiên thì model "nhìn thấy tương lai".

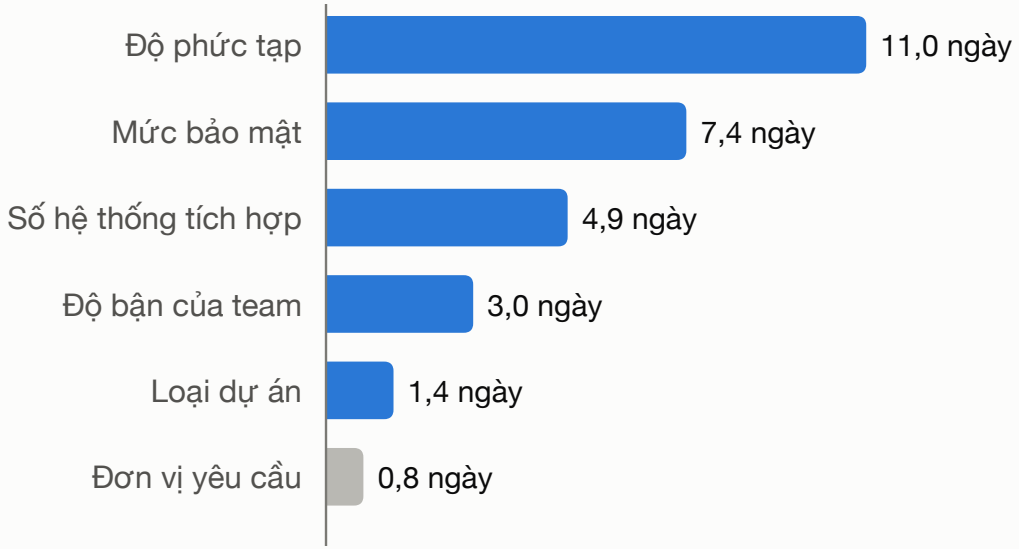
Cycle time	MAE (ngày)	R^2
Mốc so sánh	31,8	-0,00
Linear	30,1	0,11
Gradient boosting	30,2	0,10

MAE: sai trung bình bao nhiêu ngày. R^2: phần biến động model giải thích được (1 là hoàn hảo). Rework ở bước kiểm thử: ROC-AUC 0,64 (0,5 là đoán bừa).

Ý chính: Model yếu, và điều đó là bình thường. Phần lớn biến động đến từ hàng đợi và số lần làm lại ngẫu nhiên, không phải từ thuộc tính dự án.

ML: kết quả là thứ hạng nguyên nhân (SHAP)

Mỗi thuộc tính làm cycle time thay đổi trung bình bao nhiêu ngày



SHAP là gì

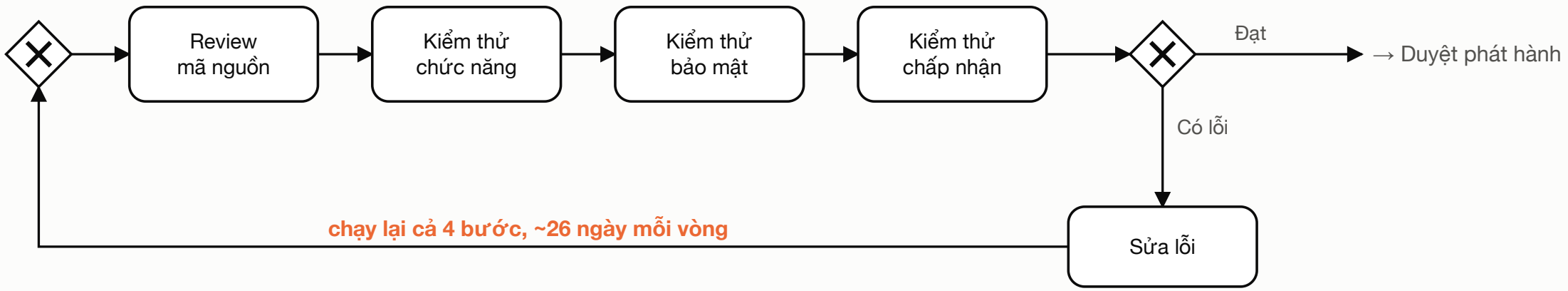
Với mỗi dự đoán, SHAP tách ra mỗi thuộc tính đẩy kết quả lên hay xuống bao nhiêu. Lấy trung bình độ lớn trên tập test để xếp hạng.

- ML tìm đúng thứ tự đã cài: độ phức tạp, mức bảo mật, rồi số tích hợp
- Biến đối chứng (đơn vị yêu cầu) đứng **cuối**. Tốt
- Kết quả này dẫn tới thay đổi C3

ML tìm lại đúng thứ ta đã cài vào dữ liệu, nên đây là phép kiểm tra phương pháp, chưa phải phát hiện mới về quy trình thật.

Ý chính: Vai trò của ML ở đây là chẩn đoán (vì sao chậm), không phải tối ưu.

Vòng kiểm thử hiện tại: 4 bước nối tiếp, có lỗi thì chạy lại cả 4

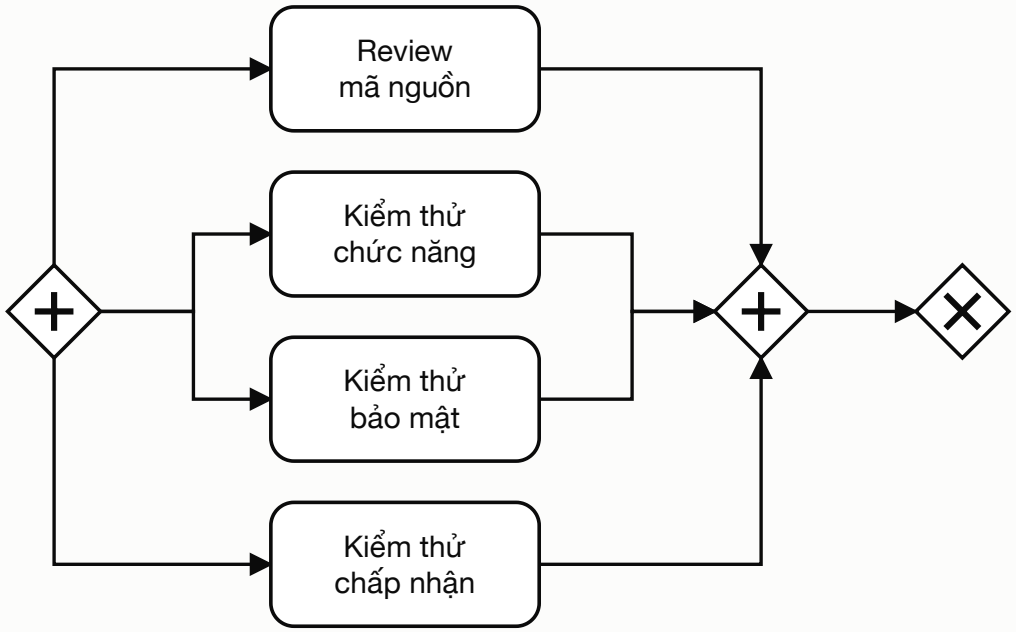


- Mỗi vòng ≈ 18 ngày làm + ~8 ngày chờ QC ≈ **26 ngày**
- 37,3% dự án phải chạy lại ít nhất 1 vòng, dù lỗi chỉ nằm ở một phần nhỏ

Ý chính: Hai vấn đề cùng lúc: mỗi vòng dài vì nối tiếp, và lặp lại vòng thì đắt.

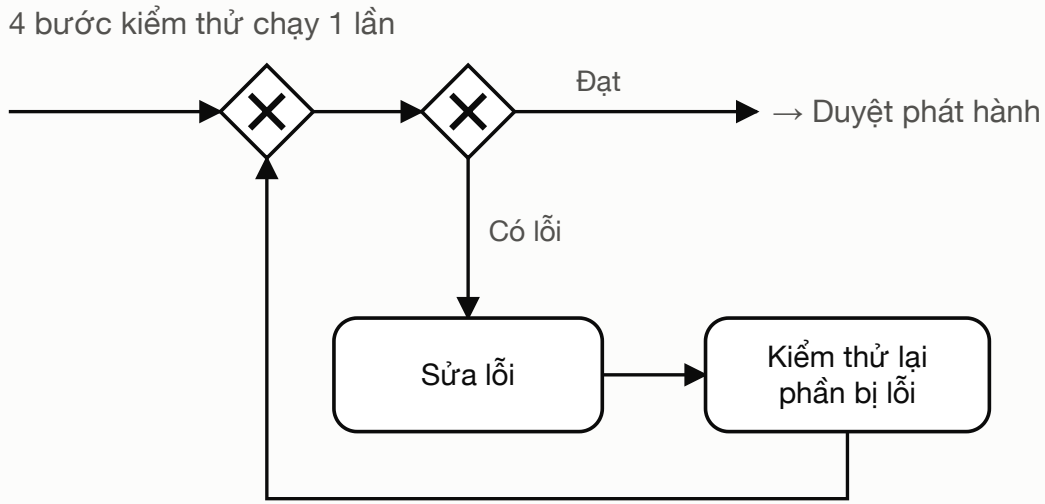
Thiết kế lại: C1 làm vòng ngắn lại, C2 làm việc lặp lại rẻ đi

C1: chạy 4 bước song song (cổng AND)



vòng test = bước dài nhất + chờ

C2: sửa xong chỉ kiểm thử lại phần bị lỗi



chỉ test lại phần sửa, ~2 ngày

Ý chính: C1 và C2 cùng nhắm vào vòng kiểm thử, nên gộp lại không được cộng đủ phần của từng cái.

4 thay đổi, mỗi cái có lý do và cái giá

	Thay đổi	Nhắm vào	Heuristic (sách, Appendix A)	Cái giá
C1	4 bước kiểm thử chạy song song	Vòng test dài vì nối tiếp	Parallelism	Chi phí ↑, chất lượng ↓: 4 tester cùng làm 1 dự án
C2	Chỉ kiểm thử lại phần bị lỗi	Mỗi lần sửa phải test lại cả vòng	Activity elimination	Chất lượng ↓: lỗi ở phần không test lại có thể lọt qua
C3	Phân tích yêu cầu sâu hơn cho dự án rủi ro cao	Độ phức tạp, mức bảo mật (từ ML)	Triage	Chi phí ↑, thêm ~1,6 ngày trung bình
C4	Thêm 1 người QC (8 → 9)	QC bận 87%	Extra resources	Chi phí ↑: thêm 1 suất lương

- C1, C2 sửa file BPMN (`to_be_c1.bpmn`, `to_be_c2.bpmn`). C3, C4 chỉ đổi tham số
- Mỗi thay đổi được đánh giá theo **Devil's Quadrangle**: thời gian, chi phí, chất lượng, linh hoạt. Nhanh hơn luôn phải trả giá ở đâu đó

Ý chính: Máy không tự chọn thiết kế. Con người chọn dựa trên nguyên nhân, rồi mô phỏng kiểm chứng.

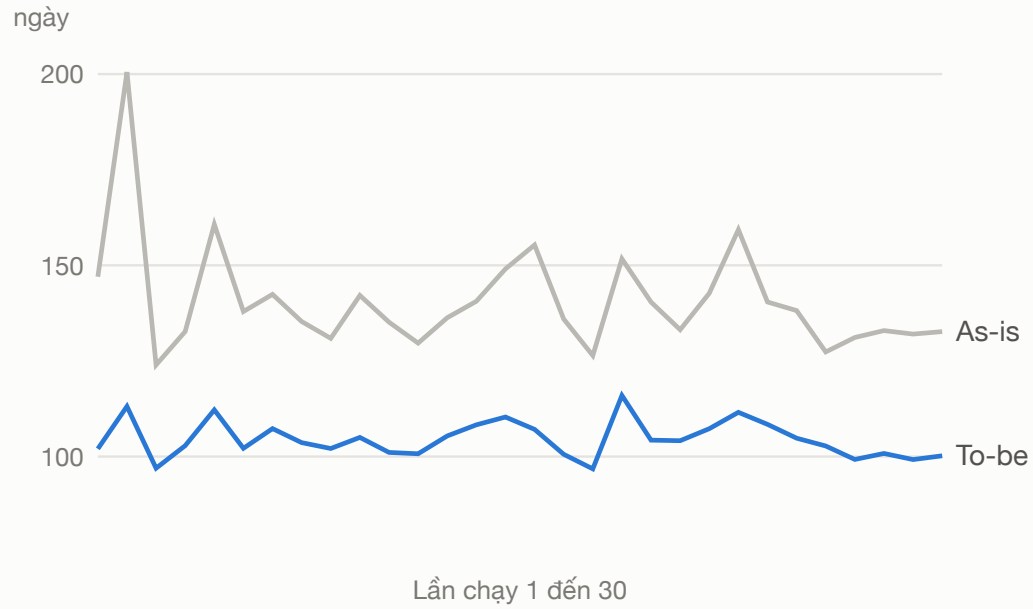
Đánh giá: so sánh công bằng giữa các kịch bản

- Chạy đủ **2^4 = 16 kịch bản**: từ as-is đến đủ 4 thay đổi
- Mỗi kịch bản chạy **30 lần**, mỗi lần 2.000 dự án
- Lần chạy thứ r của mọi kịch bản dùng **cùng chuỗi số ngẫu nhiên**: cùng dự án đến, cùng thuộc tính, cùng "may rủi"
- Phần tiết kiệm được tính **theo từng cặp** (as-is lần r so với to-be lần r), rồi tính khoảng tin cậy 95% bằng phân phối t

Vì sao dùng chung số ngẫu nhiên

Lần nào as-is gặp nhiều dự án khó thì to-be cũng gặp đúng những dự án đó. Hiệu ứng may rủi triệt tiêu, chỉ còn lại khác biệt do thiết kế.

Cycle time trung bình mỗi lần chạy (ngày)

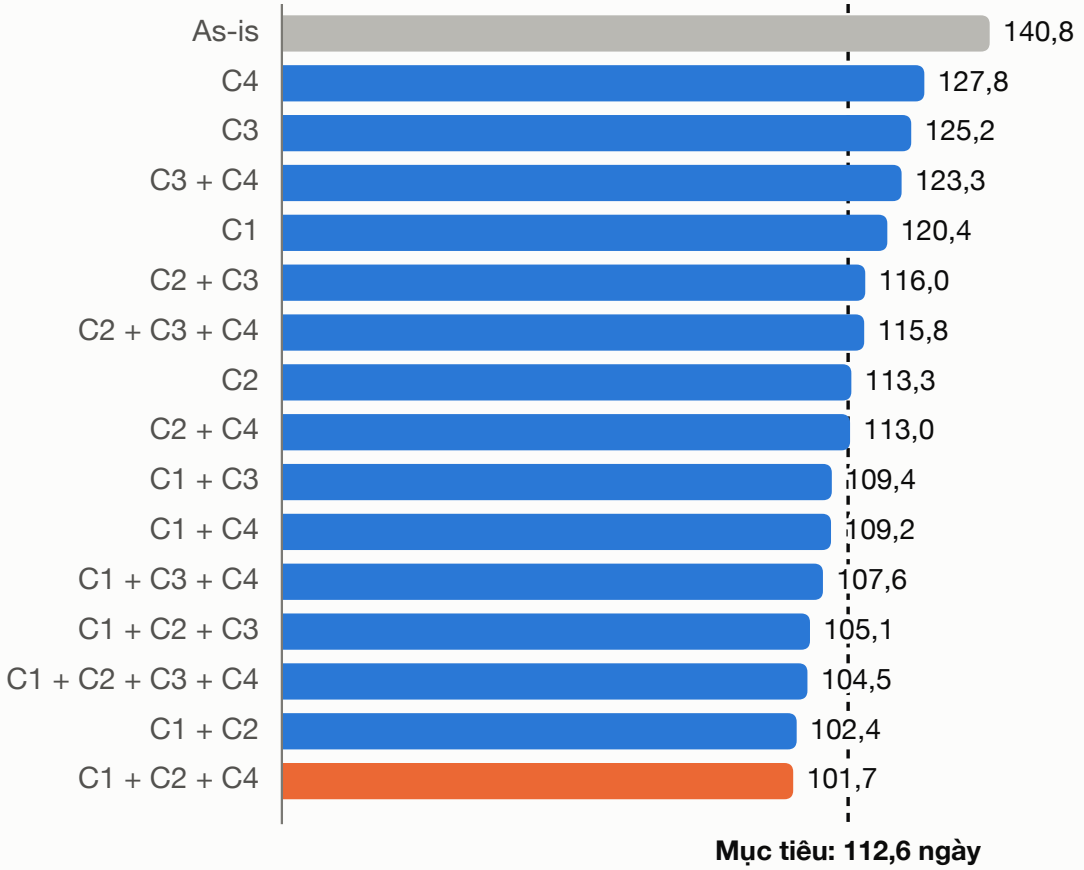


Hai đường lên xuống cùng nhau: lần chạy nào as-is chậm thì to-be cũng chậm. Khoảng cách giữa hai đường gần như không đổi, và đó là phần thiết kế mang lại.

Ý chính: Common random numbers giúp so sánh chính xác hơn với cùng số lần chạy.

Kết quả: thiết kế tốt nhất là C1 + C2 + C4

Cycle time trung bình của 16 kịch bản (ngày, trung bình 30 lần chạy)



140,8

ngày, as-is

101,7

ngày, C1 + C2 + C4: nhanh hơn 27,8% (CI 95%: 24,7% đến 30,8%)

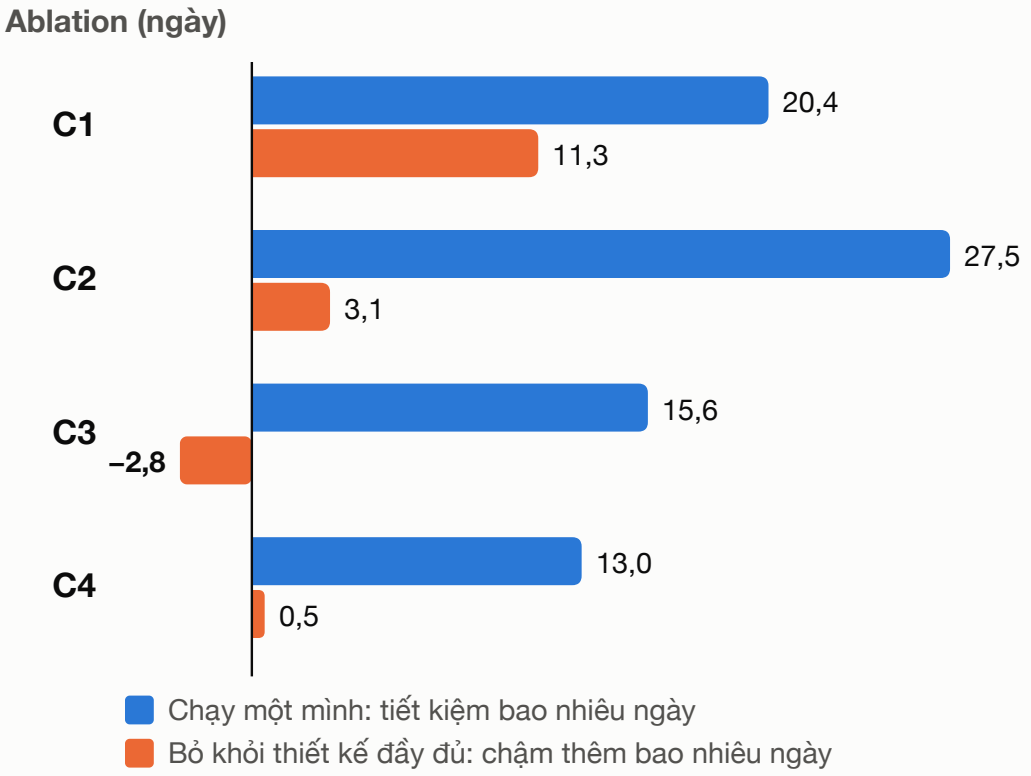
104,5

ngày, đủ 4 thay đổi: nhanh hơn 25,7% (CI 95%: 22,7% đến 28,8%)

Cả khoảng tin cậy đều nằm trên mục tiêu 20%, nên **đạt mục tiêu**.

Ý chính: Mục tiêu là 80% của 140,8 ngày = 112,6 ngày. Mọi kịch bản có C1 và C2 đều vượt qua.

Vì sao các thay đổi không cộng dồn



Cộng 4 thay đổi chạy riêng: 20 + 28 + 16 + 13 ≈ **76 ngày**. Bật cả 4 chỉ được **36 ngày**.

- Cả 4 nhắm vào **cùng một chỗ**: vòng kiểm thử và hàng đợi QC. Chỗ đó chỉ có chừng ấy thời gian để cắt
- C2 làm việc kiểm thử lại chỉ còn ~2 ngày. Sau đó, giảm số lần làm lại (C3) gần như không còn tiết kiệm được gì
- Nhưng C3 vẫn bắt dự án phức tạp tốn thêm ngày ở phân tích yêu cầu. Trong thiết kế đầy đủ, **C3 lỗ 2,8 ngày**
- C1 đóng góp nhiều nhất trong thiết kế đầy đủ: bỏ C1 thì chậm thêm 11,3 ngày

Ý chính: Hàng đợi không tuyến tính: khi QC bận gần 100%, bớt một chút việc thì thời gian chờ giảm rất nhiều. Vì vậy C2 mạnh hơn mức tính tay.

Đọc kết quả cho đúng: giới hạn của nghiên cứu

Khoảng tin cậy chỉ đo nhiễu của mô phỏng

"Nhanh hơn 27,8% (CI 24,7–30,8%)" nghĩa là: **với các giả định này**, chạy lại bao nhiêu lần thì kết quả vẫn nằm trong khoảng đó.

CI **không** đo khả năng giả định sai. Nếu kiểm thử lại thực tế mất 5 ngày chứ không phải 2, CI không cho biết điều đó.

Nên bảo vệ cái gì

Đừng bảo vệ con số chính xác. Hãy bảo vệ **hướng** (song song và kiểm thử ít lại thì nhanh hơn) và **điều kiện** để kết luận còn đúng.

Các mối đe doạ đến tính hợp lệ

- **Dữ liệu giả:** ML phần nào học lại chính giả định ta cài vào
- **Tham số là ước lượng**, không phải số đo
- **Model ML yếu** ($R^2 \approx 0{,}1$): dùng để xếp hạng nguyên nhân, không dùng để dự đoán từng dự án
- **Thời gian chờ thấp hơn thực tế:** không có trễ bàn giao, nghỉ lễ, đa nhiệm
- **Mức giảm lỗi của C3 là giả định**
- Mô hình to-be có 31–33 phần tử, hơi vượt mức 30 của 7PMG (G7)

Ý chính: Nói rõ giới hạn trước khi người nghe hỏi. Điều đó làm kết quả đáng tin hơn.

Bước tiếp theo đề xuất

1. Phân tích độ nhạy (sensitivity analysis)

Thay đổi các giả định quan trọng nhất, rồi xem kết luận có còn đúng không:

- Thời gian kiểm thử lại (1 → 6 ngày)
- Tỷ lệ phát hiện lỗi
- Số yêu cầu mỗi năm
- Mức giảm lỗi của C3

Mục tiêu là một câu kiểu: **"thiết kế vẫn đạt 20% chừng nào kiểm thử lại còn dưới X ngày"**. Ngân hàng tự kiểm tra được câu đó.

2. Đưa ML vào trong quy trình (C5)

Hiện ML nằm ngoài quy trình, chỉ để phân tích. Đề xuất: khi yêu cầu đến, model dự đoán rủi ro bị trả về. Chỉ dự án rủi ro cao mới đi qua bước phân tích sâu.

Mô phỏng C5 (phân loại bằng model) cạnh C3 (phân loại bằng quy tắc). Nếu C5 tốt hơn, ML có đóng góp đo được vào kết quả cuối.

Ngoài phạm vi đồ án: lấy log thật từ Jira hoặc Git và dùng process mining thay cho giả định. Đề bài không cho dùng dữ liệu thật nên chưa làm.

Ý chính: Sensitivity analysis là bước đáng làm nhất. Nó biến giả định thành điều kiện có thể kiểm tra.

Thuật ngữ

Cycle time	Thời gian từ lúc nhận yêu cầu đến go-live, tính bằng ngày làm việc
Cycle time efficiency	Thời gian xử lý chia cho cycle time
Lane, resource pool	Một team có số người cố định. Hết người rảnh thì việc phải chờ
Discrete-event simulation	Mô phỏng nhảy từ sự kiện này sang sự kiện kế
Biến đối chứng	Thuộc tính cố tình không ảnh hưởng gì (đơn vị yêu cầu)
MAE / R^2 / ROC-AUC	Sai trung bình / phần biến động giải thích được / khả năng xếp đúng dự án rủi ro (0,5 là đoán bừa)
Devil's Quadrangle	4 trục đánh đổi: thời gian, chi phí, chất lượng, linh hoạt
Ablation	Bật hoặc tắt từng thay đổi để đo đóng góp của nó
Processing / waiting time	Thời gian đang được làm / thời gian chờ người rảnh
Rework	Bị trả về để làm lại một phần
Event log	Bảng ghi mỗi bước của mỗi dự án: bắt đầu khi nào, kết thúc khi nào, ai làm
Planted effect	Hiệu ứng cố tình cài vào dữ liệu giả, để kiểm tra ML
SHAP	Đo mỗi thuộc tính đẩy dự đoán lên hay xuống bao nhiêu
Heuristic	Mẫu cải tiến quy trình có sẵn trong sách (Appendix A)
Common random numbers	Mọi kịch bản dùng chung chuỗi số ngẫu nhiên
Khoảng tin cậy 95%	Khoảng mà kết quả gần như chắc chắn nằm trong, nếu giả định đúng